Министерство высшей науки и образования РФ

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №7 по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнил: ст. гр. 25ВВВ2

Тургенев И.В.

Козлов А.В.

Проверил: к.т.н. доцент

Юрова О.В.

к.т.н. доцент

Деев М.В.

Пенза 2026

Название

Обход графа в глубину

Цель работы:

Изучение работы метода обхода графа в глубину

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

Описание использованных методов для решения заданий

Задание 1:

#include <stdio.h> подключает функции стандартного ввода-вывода printf и scanf. #include <stdlib.h> — функции выделения и освобождения динамической памяти malloc и free, а также генератор псевдослучайных чисел rand и srand. #include <time.h> — функцию time для инициализации генератора случайных чисел текущим временем. #include <stdbool.h> — логический тип данных bool со значениями true и false.

Функция main() запрашивает у пользователя количество вершин n. Память под матрицу смежности G размером n × n выделяется динамически как массив указателей на строки. Генератор случайных чисел инициализируется вызовом srand(time(NULL)). Матрица смежности заполняется для неориентированного графа: на главной диагонали устанавливаются нули G[i][i] = 0 (отсутствие петель), а по верхнему треугольнику (j = i + 1) рёбра генерируются случайным

образом rand() % 2 с зеркальным копированием G[j][i] = G[i][j] для обеспечения симметрии. Затем сгенерированная матрица выводится на экран вложенными циклами.

Процедура поиска в глубину реализована двумя функциями: POG_recursive и DFS_recursive. В функции POG_recursive динамически выделяется логический массив меток NUM размера n, все элементы которого инициализируются значением false («не посещена»). В цикле от 0 до n – 1 проверяется статус каждой вершины: если вершина не посещена, для неё вызывается рекурсивная процедура DFS_recursive. Это обеспечивает обход всех компонент связности графа. В функции DFS_recursive текущая вершина помечается как посещенная (NUM[v] = true) и её номер выводится на экран (со смещением +1 для нумерации от 1 до n. Далее в цикле просматривается строка матрицы смежности, и при обнаружении смежной непосещенной вершины (G[v][i] == 1 && !NUM[i]) осуществляется рекурсивный вызов DFS_recursive(i).

Метод решения — генерация случайной симметричной матрицы смежности неориентированного графа и реализация классического рекурсивного алгоритма поиска в глубину (DFS) с использованием массива меток посещенных вершин.

Задание 2:

Библиотеки используются те же. Для устранения рекурсии вызовы заменяются явным программным стеком, имитирующим системный стек вызовов.

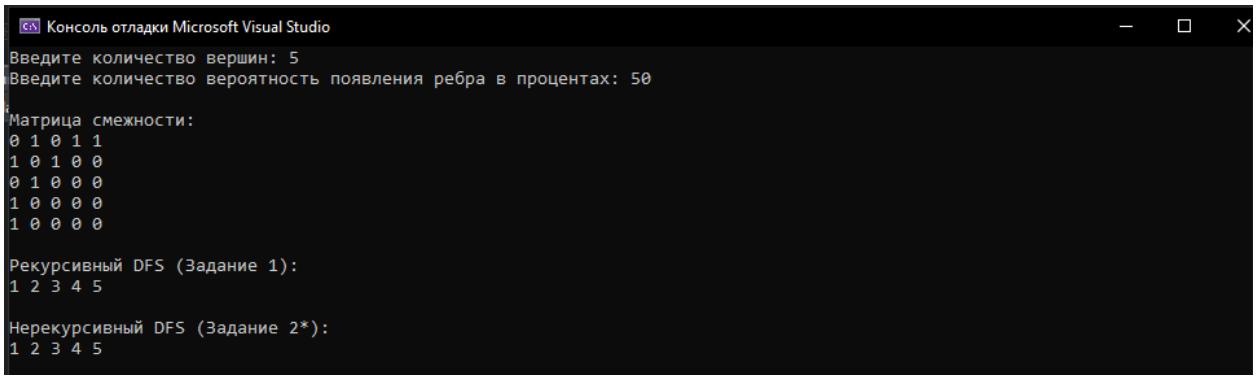
Подготовительная функция POG_iterative работает аналогично первому заданию: инициализирует массив посещенных вершин NUM значениями false и запускает нерекурсивный обход DFS_iterative для каждой еще не посещенной вершины.

В функции DFS_iterative создается динамический массив stack размера n, выполняющий роль структуры данных «стек» (LIFO), с указателем на вершину стека top = -1. Начальная вершина помещается в стек (stack[++top] = start), помечается посещенной в массиве NUM и выводится на экран. Основной цикл выполняется до тех пор, пока стек не пуст (top >= 0). На каждой итерации

рассматривается вершина на вершине стека (v = stack[top]). С помощью цикла просматриваются ее соседи: как только находится первая смежная непосещенная вершина i, она помечается как посещенная, ее номер выводится на экран, она помещается на вершину стека (stack[++top] = i), флаг found устанавливается в true, а цикл поиска прерывается оператором break (переход на шаг вглубь). Если у текущей вершины не найдено ни одного непосещенного соседа (!found), происходит возврат назад (backtracking) путем удаления текущей вершины из стека (top--). По окончании работы функции память из-под стека и массива меток освобождается через free().

Метод решения — преобразование рекурсивного алгоритма обхода в глубину в итеративный с помощью явного моделирования стека состояний и механизма отката назад при достижении тупиковых вершин.

Задание 1/2:



Выводы

В ходе выполнения лабораторной работы были изучены и освоены метод обхода графа – в глубину.